

No-Fuckery Checklist

TOOLCHAIN

CMake + VS Code

TARGET

Pi 5 / wiringPi

SCOPE

per-lesson routine

Every item here traces back to a real failure from setting this project up. Work through the relevant section each time you touch the build — **this isn't theory, it's a rap sheet.**

BEFORE Creating a new lesson folder

Do these in order, before you touch the top-level `CMakeLists.txt`.

- ☐ Folder + `main.cpp` exist first — fully — before editing the top-level file
`add_subdirectory(03_thing)` pointed at a folder that doesn't exist yet fails instantly. Create and populate the folder, *then* wire it into the parent.
- ☐ Type the CMake files by hand — never paste **CRITICAL**
Every non-logic failure so far traced back to something pasted: a `cat << EOF` block landing inside a file, or an invisible character riding along in a copy. Two lines per lesson is cheap enough to type.
- ☐ Same target name, both lines, character-for-character
`add_executable(0N_name main.cpp)`
`target_link_libraries(0N_name PRIVATE ${WIRINGPI_LIB})`
A mismatched name here is a silent trap — zero vs. capital-O, stray space — that `cmake` reports as "target not built by this project."

EVERY SWITCH

Moving to a different lesson

VS Code does not follow your open file. This is the one that actually cost real time.

- ☐ Set the active build target explicitly CRITICAL

`Ctrl+Shift+P` → "CMake: Set Build Target" → pick the lesson you're actually editing. VS Code stays on the last-selected target — a clean build of the wrong target looks identical to success.

- ☐ Glance at the status bar target name before hitting build

A two-second check that catches the mismatch before you waste a build cycle wondering why "nothing changed."

BEFORE TRUST

Trusting that a build succeeded

"Build succeeded" and "the thing I meant to test is correct" are two different claims.

- ☐ Confirm success refers to the target you meant

If unsure which target just built, re-check Phase 2 first — don't assume.

- ☐ Compiling clean $\neq$ correct — check pin-numbering scheme CRITICAL

`wiringPiSetup()` vs. `wiringPiSetupGpio()` compiles fine either way — it's a silent numbering bug, not a build error. This project standardized on `wiringPiSetupGpio()` (BCM numbering) everywhere. Confirm every new lesson uses it.

OCCASIONAL**When something looks wrong**

Not every-time checks — reach for these only when a specific symptom shows up.

☐ Cryptic CMake parse error → check for hidden characters

Run `cat -A path/to/CMakeLists.txt` before assuming the logic is wrong. Reveals stray tabs, non-breaking spaces, or line-ending artifacts a paste can leave behind invisibly.

☐ Don't run manual `cmake ..` alongside VS Code's own build dir

CMake Tools manages its own `out/build/...` directory via `CMakePresets.json`. A separate manually-created `build/` folder from an earlier terminal session can sit alongside it and cause confusion about which one is current. Let VS Code own the build directory.

☐ CMake status bar items missing entirely

Usually cosmetic, not functional — the command palette (`Ctrl+Shift+P` → "CMake: ...") still works. Try `Ctrl+Shift+P` → "Developer: Reload Window" to restore the buttons.